

Module 5

Scope and benefits of test automation, Design and implementation of automation frameworks, Utilization of testing tools for automation, Object-oriented testing principles, Analysis of software quality metrics and continuous improvement initiatives.

WHAT IS TEST AUTOMATION?

- Developing software to test the software is called test automation.
- Test automation can help address several problems.
- The time thus saved can be used effectively for test engineers to
 1. Develop additional test cases to achieve better coverage;
 2. Perform some specialized tests like ad hoc testing
 3. Perform some extra manual testing.
- The time saved in automation can also be utilized to develop additional test cases, thereby improving the coverage of testing.

- Test automation is the process of using specialized software tools or frameworks to automatically execute pre-defined test cases on a software application.
- Unlike manual testing, where a tester executes tests manually, test automation allows for faster and more efficient testing by running scripts that simulate user interactions with the application.
- The goal is to increase the speed, coverage, and effectiveness of the testing process.
- Test automation is commonly used for regression testing, performance testing, and repetitive tasks that need to be repeated frequently.
- It can be applied at various levels, such as unit tests, integration tests, system tests, and acceptance tests.

TERMS USED IN AUTOMATION

- We have been using the term “test case”, a test case is a set of sequential steps to execute a test operating on a set of predefined inputs to produce certain expected outputs.
- There are two types of test cases—automated and manual. As the names suggest, a manual test case is executed manually while an automated test case is executed using automation.
- A test case (manual or automated) can be represented in many forms. It can be documented as a set of simple steps, or it could be an assertion statement or a set of assertions.
- An example of an assertion is “Opening a file, which is already opened should fail.” An assertion statement includes the expected result in the definition itself.

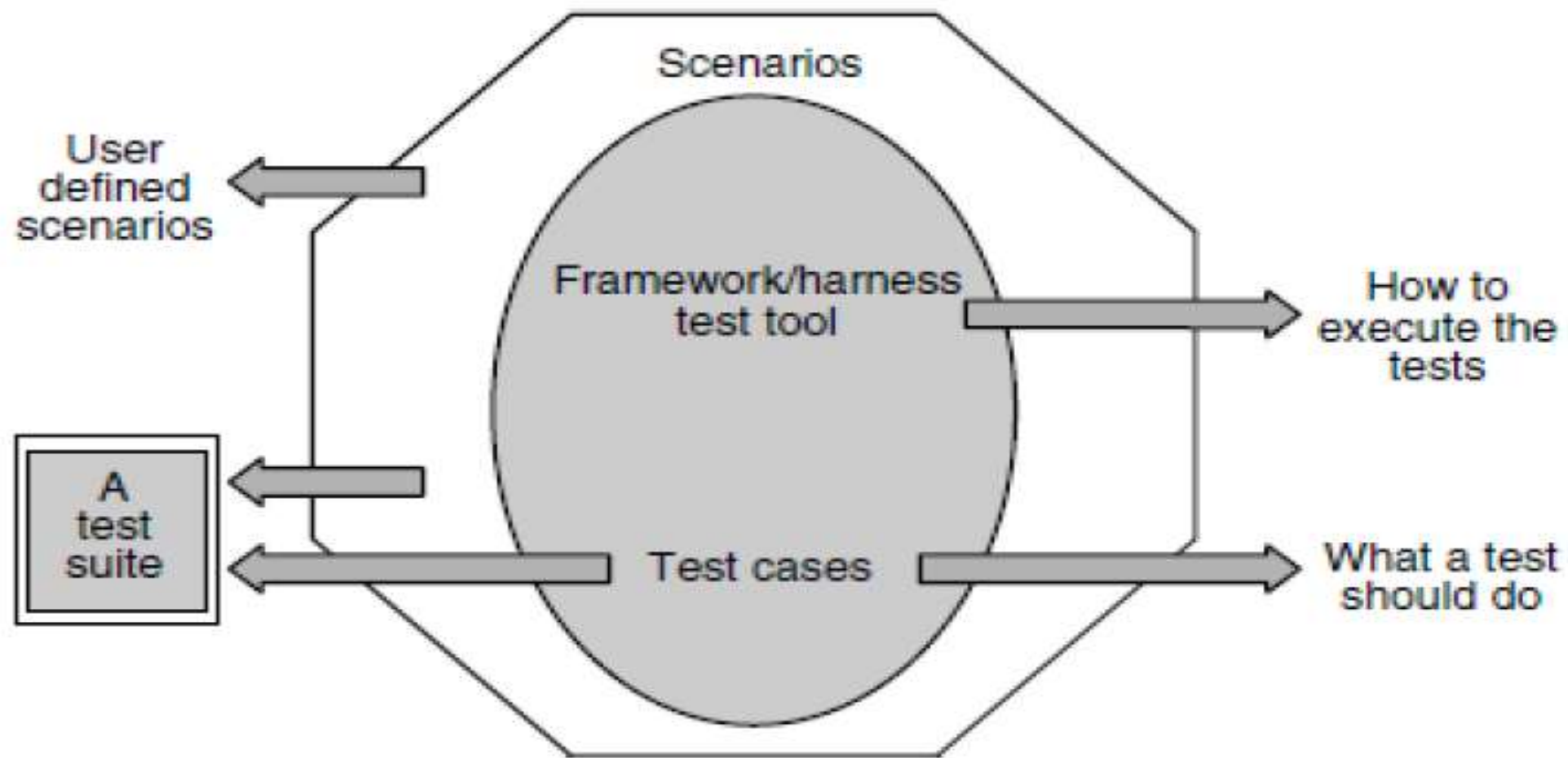


Figure 16.1 Framework for test automation

- Testing involves several phases and several types of testing. Some test cases are repeated several times during a product release because the product is built several times.
- Not only are the test cases repetitive in testing, some operations that are described as steps in the test cases too are repetitive.
- The framework-/test tool-specific requirements are not just for test cases or products. They are generic requirements for all products that are being tested in an organization.
- The automation belief is based on the fact that product operations (such as log in) are repetitive in nature and by automating the basic operations and leaving the different scenarios (how to test) to the framework/test tool, great progress can be made.
- This ensures code re-use for automation and draws a clear boundary between “what a test suite has to do” and “what a framework or a test tool should complement.” When scenarios are combined by basic operations of the product, they become automated test cases.

- When a set of test cases is combined and associated with a set of scenarios, they are called “test suite.”
- A Test suite is nothing but a set of test cases that are automated and scenarios that are associated with the test cases.

WHAT TO AUTOMATE, SCOPE OF AUTOMATION

- The first phase involved in product development is requirements gathering; it is no different for test automation as the output of automation can also be considered as a product (the automated tests).
- The automation requirements define what needs to be automated looking into various aspects.
- The specific requirements can vary from product to product, from situation to situation, from time to time.

1. Identifying the Types of Testing Amenable to Automation- Certain types of tests automatically lend themselves to automation.

Stress, reliability, scalability, and performance testing - These types of testing require the test cases to be run from a large number of different machines for an extended period of time, such as 24 hours, 48 hours, and so on.

- It is just not possible to have hundreds of users trying out the product day in and day out—they may neither be willing to perform the repetitive tasks, nor will it be possible to find that many people with the required skill sets.
- Test cases belonging to these testing types become the first candidates for automation.
- **Regression tests** - Regression tests are repetitive in nature. These test cases are executed multiple times during the product development phases. Given the repetitive nature of the test cases, automation will save significant time and effort in the long run.

- Functional tests - These kinds of tests may require a complex set up and thus require specialized skill, which may not be available on an ongoing basis.
- Automating these once, using the expert skill sets, can enable using less-skilled people to run these tests on an ongoing basis.
- In the product development scenario, a lot of testing is repetitive as a good product can have a long lifetime if the periodic enhancements and maintenance releases are taken into account. This provides an opportunity to automate test cases and execute them multiple times during release cycles.

2. Automating Areas Less Prone to Change- In a product scenario, the changes in requirements are quite common. In such a situation, what to automate is easy to answer. Automation should consider those areas where requirements go through lesser or no changes. Normally change in requirements cause scenarios and new features to be impacted, not the basic functionality of the product.

3. Automate Tests that Pertain to Standards- One of the tests that products may have to undergo is compliance to standards.

For example, a product providing a JDBC interface should satisfy the standard JDBC tests. These tests undergo relatively less change. Even if they do change, they provide backward compatibility by which automated scripts will continue to run.

- **4 Management Aspects in Automation-** Prior to starting automation, adequate effort has to be spent to obtain management commitment.
- Automation generally is a phase involving a large amount of effort and is not necessarily a one-time activity.
- The automated test cases need to be maintained till the product reaches obsolescence.
- Since it involves significant effort to develop and maintain automated tools, obtaining management commitment is an important activity.
- Since automation involves effort over an extended period of time, management permissions are only given in phases and part by part.

Scope of Test Automation

- Test Case Execution: Automation is primarily used to execute predefined test cases automatically.
- Regression Testing: Repeating tests to ensure new changes haven't broken existing functionality.
- Continuous Integration/Continuous Deployment (CI/CD): Automating tests within CI/CD pipelines to ensure that every code change is tested quickly.
- Performance Testing: Automating load, stress, and performance tests to evaluate the application under different conditions.
- Cross-Platform Testing: Automating tests across multiple browsers, devices, or operating systems.
- Test Coverage: Enhancing test coverage by automating scenarios that would be tedious or impossible to test manually.

Benefits of Test Automation

- **Speed and Efficiency:** Automated tests can run much faster than manual tests, especially when the same test needs to be executed multiple times across different environments. This reduces the time required to perform testing, accelerating the development process.
- **Reusability:** Once created, automated test scripts can be reused across different versions of the application. This is particularly helpful for regression testing, where you need to rerun the same tests with every new release.
- **Consistency and Accuracy:** Automated tests perform the exact same steps each time, eliminating human errors like missed steps or incorrect data entry that can occur in manual testing. This leads to more accurate and reliable test results.
- **Cost Efficiency in the Long Run:** While creating automation scripts requires an initial investment in terms of time and resources, the long-term cost savings are significant as it reduces the amount of manual testing needed. More tests can be performed in less time, reducing the overall time to market.

- **Faster Feedback:** Automated tests can be executed frequently, especially within CI/CD pipelines, providing quick feedback to developers on the health of the application after each change.
- **Better Test Coverage:** Automated testing allows you to run a large number of tests in a short period, improving test coverage, including edge cases and complex scenarios that would be difficult to test manually.
- **Support for Agile and DevOps Practices:** Test automation integrates well into agile and DevOps practices, where rapid iterations and frequent deployments are crucial. Automated tests can be triggered every time new code is checked in, ensuring that the application remains stable as it evolves.

DESIGN AND ARCHITECTURE FOR AUTOMATION

- Design and architecture is an important aspect of automation. As in product development, the design has to represent all requirements in modules and in the interactions between modules.
- the thin arrows represent the internal interfaces and the direction of flow and thick arrows show the external interfaces.
- All the modules, their purpose, and interactions between them are described in the subsequent sections.
- Architecture for test automation involves two major heads: a test infrastructure that covers a test case database and a defect database or defect repository.

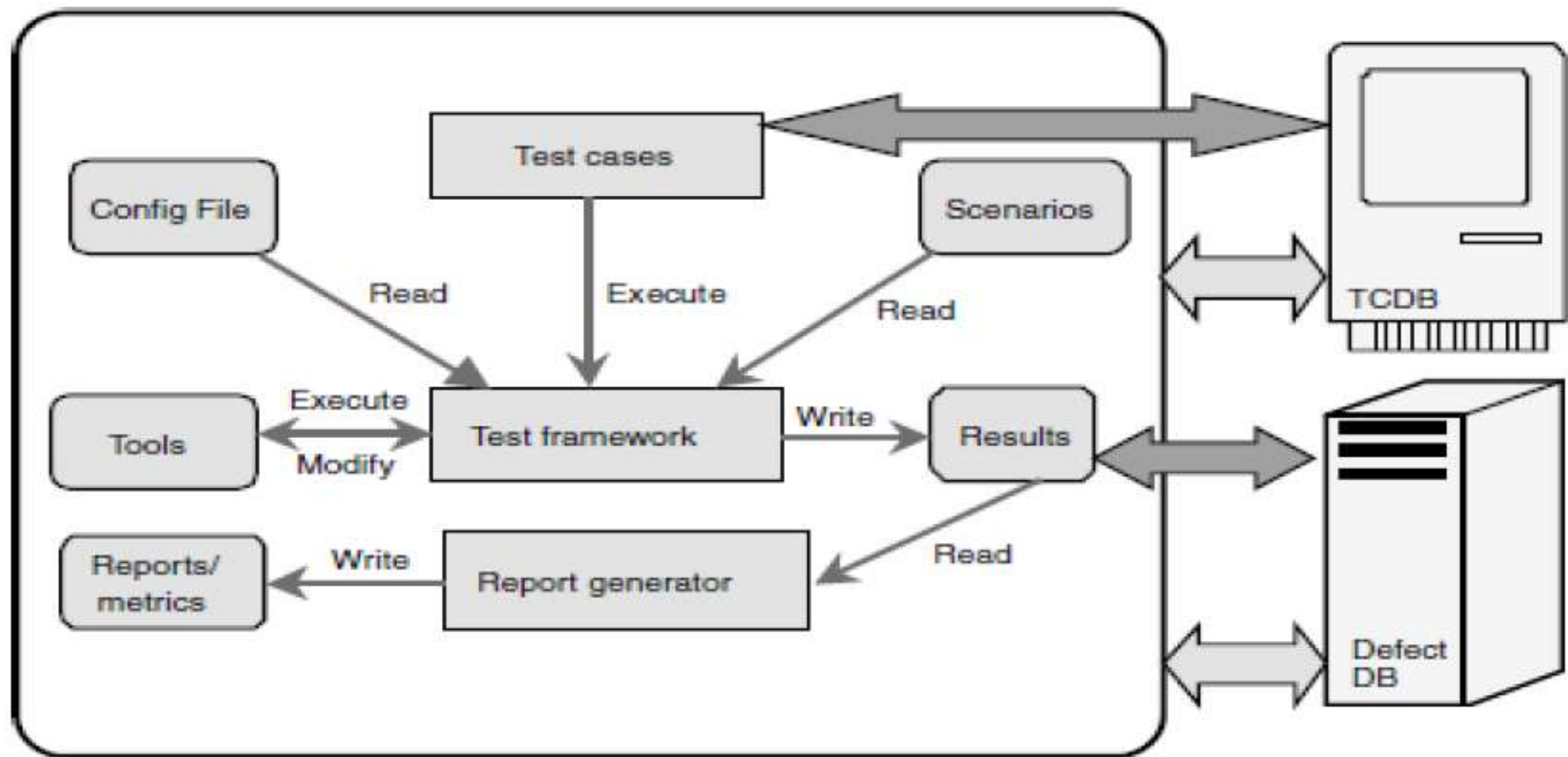


Figure 16.2 Components of test automation.

1. External Modules

- There are two modules that are external modules to automation— TCDB and defect DB.
- To recall, all the test cases, the steps to execute them, and the history of their execution (such as when a particular test case was run and whether it passed/failed) are stored in the TCDB.
- The test cases in TCDB can be manual or automated. The interface shown by thick arrows represents the interaction between TCDB and the automation framework only for automated test cases.
- Please note that manual test cases do not need any interaction between the framework and TCDB.

- Defect DB or defect database or defect repository contains details of all the defects that are found in various products that are tested in a particular organization.
- It contains defects and all the related information (when the defect was found, to whom it is assigned, what is the current status, the type of defect, its impact, and so on).
- Test engineers submit the defects for manual test cases.
- For automated test cases, the framework can automatically submit the defects to the defect DB during execution.

2. Scenario and Configuration File Modules

- scenarios are nothing but information on “how to execute a particular test case.”
- A configuration file contains a set of variables that are used in automation.
- The variables could be for the test framework or for other modules in automation such as tools and metrics or for the test suite or for a set of test cases or for a particular test case.
- A configuration file is important for running the test cases for various execution conditions and for running the tests for various input and output conditions and states.
- The values of variables in this configuration file can be changed dynamically to achieve different execution, input, output, and state conditions.

3. Test Cases and Test Framework Modules

- A test cases means the automated test cases that are taken from TCDB and executed by the framework. Test case is an object for execution for other modules in the architecture and does not represent any interaction by itself.
- A test framework is a module that combines “what to execute” and “how they have to be executed.”
- It picks up the specific test cases that are automated from TCDB and picks up the scenarios and executes them.
- The variables and their defined values are picked up by the test framework and the test cases are executed for those values.
- A test framework can be developed by the organization internally or can be bought from the vendor.

4 Tools and Results Modules

- When a test framework performs its operations, there are a set of tools that may be required.
- For example, when test cases are stored as source code files in TCDB, they need to be extracted and compiled by build tools. In order to run the compiled code, certain runtime tools and utilities may be required.
- For example, IP Packet Simulators or User Login Simulators or Machine Simulators may be needed. In this case, the test framework invokes all these different tools and utilities.

5 Report Generator and Reports/Metrics Modules

- Once the results of a test run are available, the next step is to prepare the test reports and metrics.
- Preparing reports is a complex and time-consuming effort and hence it should be part of the automation design.
- There should be customized reports such as an executive report, which gives very high level status; technical reports, which give a moderate level of detail of the tests run; and detailed or debug reports which are generated for developers to debug the failed test cases and the product.

Utilization of Testing Tools for Automation

- Testing tools play a crucial role in automating various types of tests in the software development lifecycle. These tools help enhance the efficiency, accuracy, and effectiveness of the testing process.
- Below is an overview of the types of testing tools used for automation, their applications, and best practices for utilizing them.

Testing of Object-Oriented Systems

- Object-oriented testing principles are guidelines that help ensure that object-oriented systems are tested effectively, focusing on the key concepts of object-oriented programming (OOP) such as encapsulation, inheritance, polymorphism, and abstraction.
- The goal is to test the system in a way that accounts for these features and ensures that all interactions between objects and components function as expected.

1. Test the Behavior of Objects (Not Just Methods)

In OOP, objects are defined by both their state (attributes) and behavior (methods). When testing, focus on how an object behaves rather than just testing individual methods in isolation.

PRIMER ON OBJECT-ORIENTED SOFTWARE

- The goal is to highlight the major concepts that cause some modifications and additions to the approach of testing that we have seen so far.
- The programming languages were algorithm-centric in that they viewed the program as being driven by an algorithm that traced its execution from start to finish
- This class of programming languages was characterized by
 1. Data being considered as separate from the operations or program
 2. Algorithm being the driver, with data being subsidiary to the algorithm.

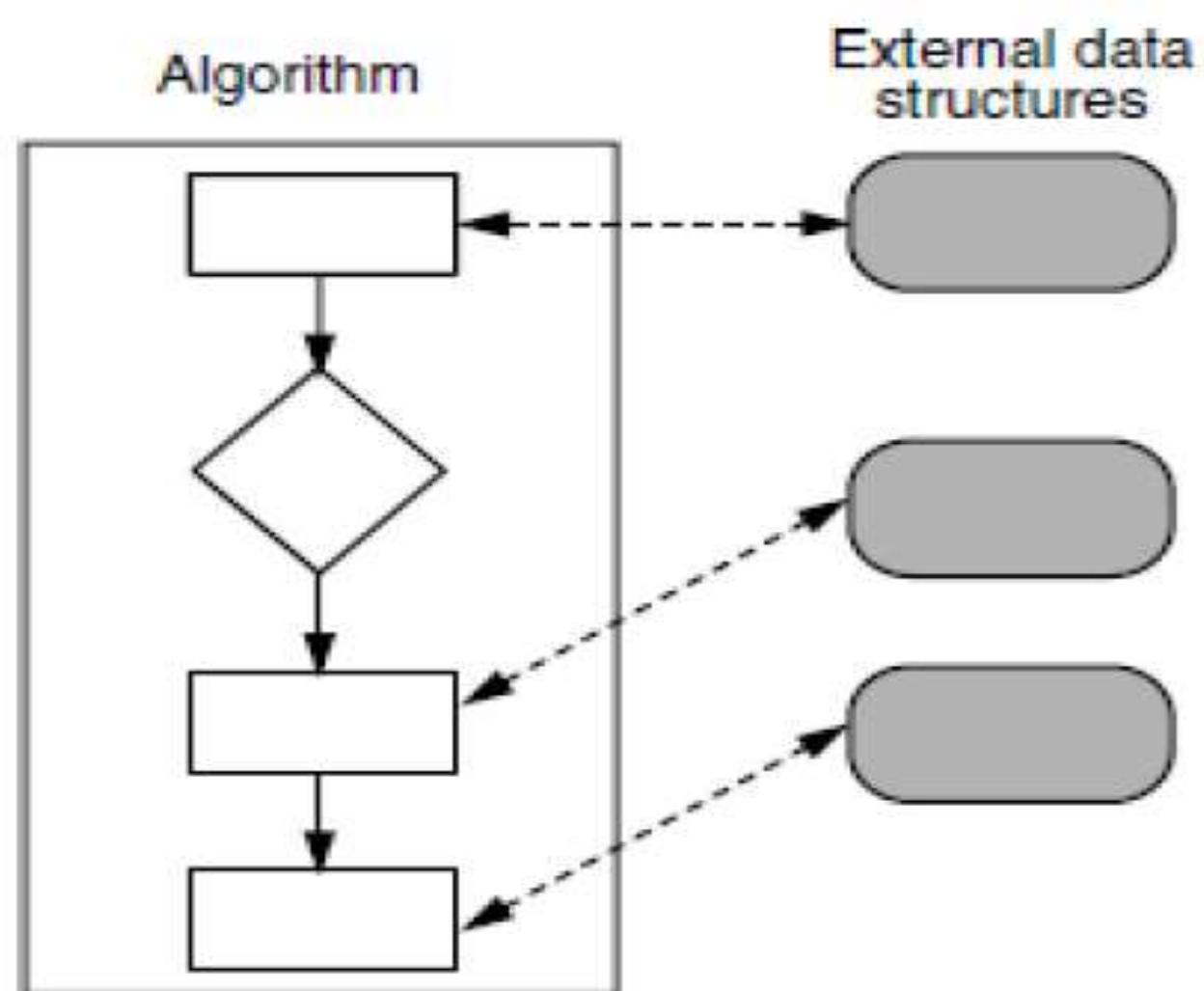


Figure 11.1 Conventional algorithm centric programming languages.

- From a testing perspective, testing a conventional procedure-oriented system therefore entailed testing the algorithm, treating data as secondary to testing the algorithm flow.
- There are two fundamental paradigm shifts in OO languages and programming: First, the language is data- or object-centric.
- There is no separation between data and the methods that operate on the data.
- The data and the methods that operate on the data go together as one indivisible unit.

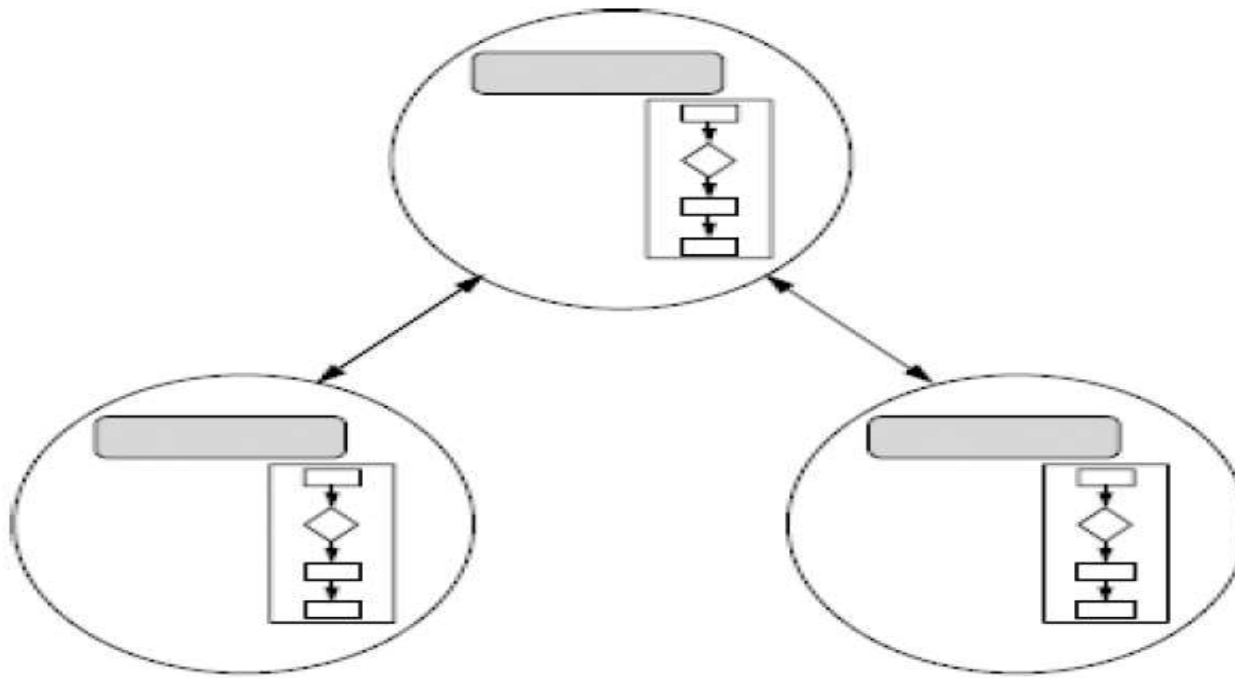


Figure 11.2 Object centric language—algorithm and data tightly coupled.

- Classes form the fundamental building blocks for OO systems.
- A class is a representation of a real-life object.
- Each class (or the real-life object it represents) is made up of attributes or variables and methods that operate on the variables.

- For example, a real-life object called a rectangle is characterized by two attributes, length and breadth. Area and perimeter are two of the operations or methods that can be performed on the rectangle object. The variables and the methods together define a rectangle class.

Analysis of software quality metrics

- When analyzing software quality metrics related exam questions, it's essential to understand that these questions typically aim to test your knowledge of various software quality indicators and how they are used to evaluate software products or processes.
- Types of Software Quality Metrics
 1. Product Metrics: These measure the quality of the software product itself.
 - Defect Density: The number of defects found per unit of size (e.g., per 1,000 lines of code).
 - Code Coverage: Percentage of the code that is executed during testing.
 - Cyclomatic Complexity: A metric that measures the complexity of a program's control flow.
 - Maintainability Index: Reflects how easy it is to maintain the software.

2. Process Metrics: These measure the efficiency and effectiveness of the software development process.

- Defect Arrival Rate: Rate at which defects are found during the development and testing phases.
- Lead Time: Time taken from starting a task until it is completed.
- Productivity Metrics: Measures like Lines of Code (LOC) per developer per day.
- Change Request Frequency: Tracks the frequency of change requests and how well the team handles them.

3. People Metrics: These track the performance of the individuals or teams involved in the development process.

- Team Velocity: The number of story points a team can handle in a sprint.
- Employee Satisfaction: Metrics on how engaged or satisfied the team is, often indirectly affecting software quality.

Quality Attributes

- Reliability: Metrics like Mean Time Between Failures (MTBF) and defect density.
- Usability: User satisfaction surveys, task success rates, and error rates.
- Performance: Response time, throughput, and resource utilization metrics.
- Security: Number of security vulnerabilities, time to fix vulnerabilities, etc.
- Scalability: The ability of the system to handle growing amounts of work.

Quality Assurance (QA) Metrics

- Test Coverage: What percentage of the code is covered by tests.
- Test Pass Rate: The percentage of tests that pass during the testing phase.
- Defect Discovery Rate: How quickly defects are identified during testing.
- Defect Resolution Time: How long it takes to fix a reported defect.
- Defect Leakage: The percentage of defects that are not detected during testing but are discovered after release.

Metric Visualization and Interpretation

- Thresholds and Benchmarks: How metrics are compared to predefined benchmarks or industry standards.
- Graphs and Charts: Understanding the use of bar charts, histograms, line graphs, and scatter plots to represent metric data.
- Root Cause Analysis: Understanding how to analyze metrics to identify the root causes of issues like high defect density or poor performance.

Challenges and Trade-offs

- Overhead of Data Collection: How collecting certain metrics might add overhead to the development process.
- Metric Overload: How focusing on too many metrics can lead to information overload and distract from important objectives.
- Misuse of Metrics: For example, using metrics like Lines of Code (LOC) as a measure of productivity may not accurately reflect software quality or team efficiency.

Continuous Improvement Initiatives

- Continuous improvement (CI) is an ongoing effort to enhance software quality and processes through incremental improvements over time. This approach is driven by analyzing data from quality metrics and making systematic changes.
- Key components of continuous improvement initiatives include:

1 . Agile Development Practices:

- Agile methodologies, such as Scrum or Kanban, focus on iterative progress and rapid feedback. Through short development cycles called sprints, teams can continuously improve product quality, incorporating changes based on customer feedback, testing results, and performance data.
- Iterative Development: Enables teams to address quality issues early in the development process by making small, incremental changes.
- Daily Standups and Retrospectives: Provide teams with regular opportunities to identify and address process inefficiencies and quality bottlenecks.
- Customer Feedback Integration: Agile teams integrate customer feedback frequently to refine features and fix issues, ensuring the software meets user expectations.

Lean Software Development:

- Lean focuses on reducing waste in the development process by continuously eliminating inefficiencies. Applying lean principles leads to:
- Faster Releases: Reducing cycle times and addressing issues faster.
- Reduced Defects: Implementing small changes that lead to fewer defects over time.
- Improved Flow: Streamlining development processes to deliver value to the customer more efficiently.

Automation in Testing and Deployment:

- Automation tools significantly aid in continuous improvement initiatives by ensuring faster and more reliable software testing and deployment.
- Automated Unit Tests: Running unit tests automatically during every build ensures issues are caught early and continuously verifies that the software functions as expected.
- Continuous Integration (CI): CI tools automatically integrate code changes into a shared repository, triggering automated builds and tests. This reduces the chances of integration issues.
- Continuous Delivery (CD): CD ensures that software is automatically deployed to production after passing tests, reducing the risk of errors during manual deployments.

Root Cause Analysis and Corrective Actions:

- Continuous improvement often includes identifying the root causes of defects and addressing them at the source.
- This can involve:
- Failure Mode and Effects Analysis (FMEA): Systematically identifying potential failure points in the software and taking proactive steps to mitigate them.
- Post-Mortem Analysis: After defects or failures, teams conduct post-mortem meetings to identify what went wrong and how processes can be improved in future iterations.
- Corrective Actions: Implementing long-term changes in the software or development processes to prevent recurring issues.

Feedback Loops:

- Regular feedback loops from various stakeholders (customers, team members, automated tools) allow for continuous refinement of processes and the software product. These loops might include:
- User Feedback: Direct feedback from customers or end-users to identify problems and improve usability.
- Internal Team Feedback: Regular team retrospectives and meetings to assess process effectiveness and identify improvement opportunities.
- Testing Feedback: Test results provide insight into the areas of the product that need more focus, whether it's performance, security, or functionality.

Kaizen (Incremental Improvements):

- Kaizen, a core principle in continuous improvement, emphasizes small, incremental changes over time rather than large, disruptive changes. This approach ensures that improvements are sustainable and that teams are always learning and adapting.

Combining Quality Metrics with Continuous Improvement

- To ensure that quality metrics lead to meaningful improvements, it is crucial to integrate them with continuous improvement efforts.
- Set Clear Goals: Define the key quality metrics that need to be improved and establish targets for those metrics (e.g., reducing defect density by 20% in the next quarter).
- Track Progress Over Time: Use software tools (e.g., Jira, SonarQube, Jenkins) to collect and monitor quality metrics in real-time. This enables teams to track progress and make informed decisions.
- Prioritize Areas for Improvement: Based on the metrics, identify the areas of the software or development process that require attention. For instance, if test pass rates are low, focus on enhancing test coverage and quality.
- Adapt and Refine: Continuously refine processes based on data. As defects are reduced or performance improves, teams should adjust their focus and iterate on new areas for improvement.